Traffic Prediction in Bogotá: A Spatio-Temporal Graph Neural Network Approach

A White Paper in collaboration with the Inter-American Development Bank and the Government of Bogotá

José María Márquez Blanco
jose.marquez.blanco@gmail.com

Martín Sevilla
martin.sevilla@outlook.com

May 17, 2026

Abstract

Traffic congestion remains one of the most critical challenges for urban planning in Latin America, particularly in high-density metropolitan areas like Bogotá. Accurate forecasting of traffic conditions is essential for optimizing traffic management systems and reducing economic and environmental costs. This white paper presents the Spatio-Temporal Graph Neural Network (ST-GNN) framework developed in partnership with the Inter-American Development Bank (IDB) and the Government of Bogotá. The proposed system leverages a comprehensive data processing pipeline that integrates geospatial, traffic, event, incident, road-state, weather, and urban context data into a unified graph-temporal representation. The core of the model architecture is based on advanced ST-GNN variants, which are designed to capture the complex spatial and temporal dependencies inherent in urban traffic networks. The paper details the data processing strategies, including sparse storage, seed-time batching, and missing data imputation, as well as the training infrastructure that supports scalable model development. The proposed sampling methodology addresses the imbalance of congestion states in the training data, ensuring that the model is exposed to a representative mix of traffic conditions. By accurately modeling the spatio-temporal dynamics of Bogotá's traffic, this system aims to provide actionable insights for proactive traffic management and contribute to the broader goal of improving urban mobility in Latin America.

1 Introduction

Urban mobility is a defining challenge of the 21st century. For cities like Bogotá, which consistently ranks among the most congested urban centers globally [1], the implications of traffic gridlock extend beyond mere inconvenience; they significantly impact economic productivity, air quality, and the overall quality of life for millions of citizens [2, 3]. In response to these challenges, the Government of Bogotá, in collaboration with the Inter-American Development Bank (IDB), has launched an initiative to modernize the city's traffic management capabilities through Artificial Intelligence (AI) and data-driven decision-making.

The core of this initiative lies in the ability to accurately predict traffic conditions. However, traffic flow is inherently complex, exhibiting dynamic dependencies in both *space* (the connec-

tivity of the road network) and *time* (historical flow patterns). Traditional time-series analysis methods, such as ARIMA, often fail to capture the spatial patterns between adjacent road segments [4]. Conversely, standard Convolutional Neural Networks (CNNs) are limited to grid-like structures and struggle with the irregular topology of a city's road network [5]. To address these limitations, we employ Spatio-Temporal Graph Neural Networks (ST-GNNs). These models represent the road network as a graph, where road segments are nodes and their connections (based on traffic flow and proximity) are edges, allowing for the simultaneous extraction of spatial and temporal features.

Recent advancements in this field have produced powerful architectures. The **GConvLSTM** and **GConvGRU** [6] integrate Graph Convolutional Networks (GCNs) with recurrent units, effectively learning sequence patterns on graph structures. Similarly, the Temporal Graph Convolutional Network (**TGCN**) [7] combines GCNs with Gated Recurrent Units (GRUs) to capture non-linear traffic dynamics. Attention mechanisms, as in the Attention Based Spatial-Temporal Graph Convolutional Network (**A3TGCN**) [8], remain an important reference point, although the current empirical comparison in this repository concentrates on GConvLSTM, GConvGRU, and TGCN.

This white paper details the deployment path for these ST-GNN architectures under Bogotá-specific data and operational constraints. By accurately modeling the spatio-temporal dependencies of the city's traffic, we aim to provide actionable insights for the *Secretaría de Movilidad*, facilitating proactive rather than reactive traffic management.

Beyond the architectural choice, the practical contribution of this work is also to include the construction of a graph-temporal data contract for Bogotá, the transformation of highly sparse mobility observations into efficient training tensors, the design of a sampler that explicitly addresses the imbalance between uncongested and congested conditions, and the creation of an evaluation path that distinguishes sparse validation performance from dense operational behavior. These engineering decisions are central to the project, because they determine which traffic states the model sees during training and how its predictions can be interpreted after deployment.

2 System Overview

The forecasting system can be understood as an end-to-end graph-temporal pipeline. Raw geospatial, traffic, event, incident, road-state, weather, and urban context data are first transformed by feature-specific processors into road-segment-level observations. These observations are merged into sparse HDF5 datasets indexed by node, time, and feature, while the road network is represented as a directed graph whose edges encode legal traffic flow and spatial proximity. The resulting processed data is consumed by `TrafficDataset`, which materializes seed-local subgraphs and temporal windows for training.

The training runtime then applies a chronological temporal split, builds one of the supported ST-GNN model families, and selects the seed-time sampler. In the current experimental line, SageMaker jobs read preprocessed data from EFS, retrieve the consolidated seed-time artifact from S3 when required, train GConvLSTM, GConvGRU, and TGCN variants, and export trained model artifacts back to S3. This separation between large processed data, sampling metadata,

and model outputs makes the system reproducible enough to compare model families while still remaining practical for large-scale training.

Inference follows a related but distinct operational path. The production inference repository reads current `*.parquet` sources and real-time data stores rather than historical HDF5 target files, constructs graph-aligned tensors for the current observational anchor timestamp, runs model inference over seed-local subgraphs, and reconstructs city-wide forecasts by combining overlapping predictions. This distinction is important because offline validation and real-time dense evaluation answer different questions: the former measures model behavior on historical sparse training targets, while the latter measures behavior over a fuller operational prediction surface.

3 Literature Review

Traffic prediction has evolved from simple statistical baselines to complex deep learning frameworks. This section categorizes these approaches into three distinct generations: statistical methods, Euclidean deep learning, and Graph Neural Networks.

3.1 A Statistical Approach

Early attempts at traffic forecasting relied heavily on statistical theory. The Autoregressive Integrated Moving Average (ARIMA) model and its variants (e.g., Seasonal ARIMA) were widely used due to their mathematical soundness [9]. The ARIMA model can be expressed as:

$$y_t = c + \phi_1 y_{t-1} + \phi_2 y_{t-2} + \cdots + \phi_p y_{t-p} + \theta_1 \epsilon_{t-1} + \theta_2 \epsilon_{t-2} + \cdots + \theta_q \epsilon_{t-q} + \epsilon_t, \tag{1}$$

where y_t is the value at time t, c is a constant, ϕ_i are the autoregressive coefficients, θ_i are the moving average coefficients, and ϵ_t is the error term. While effective for stable, short-term predictions on single road segments, these models struggle with the non-linear and stochastic nature of complex traffic networks [10].

3.2 Euclidean Deep Learning

The advent of Deep Learning introduced Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs) to the domain. RNNs, particularly Long Short-Term Memory (LSTM) units and Gated Recurrent Units (GRUs), became the standard for capturing temporal dependencies in time-series data [11, 12]. The LSTM cell can be described mathematically as:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f), \tag{2}$$

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i), \tag{3}$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C), \tag{4}$$

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t, \tag{5}$$

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o), \tag{6}$$

$$h_t = o_t \odot \tanh(C_t), \tag{7}$$

where f_t, i_t, and o_t are the forget, input, and output gates, respectively, C_t is the cell state, and h_t is the hidden state.

Hybrid models, such as CNN-LSTM, attempted to capture spatial features using 2D convolutions [13]. However, CNNs operate on Euclidean domains (grids), making them suboptimal for road networks which are naturally non-Euclidean graphs. Treating a city map as an image often results in a loss of topological information [14].

3.3 Spatio-Temporal Graph Neural Networks (ST-GNNs)

To effectively model the non-Euclidean structure of road networks, Graph Neural Networks (GNNs) were introduced [15]. Kipf and Welling [16] proposed the Graph Convolutional Network (GCN), which generalizes the convolution operation to graphs using spectral graph theory [17]. The GCN layer can be expressed as:

$$H^{(l+1)} = \sigma\left(\tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2}H^{(l)}W^{(l)}\right), \tag{8}$$

where $\tilde{A} = A + I$ is the adjacency matrix with added self-loops, $\tilde{D}$ is the degree matrix of $\tilde{A}$, $H^{(l)}$ is the input feature matrix at layer l, $W^{(l)}$ is the learnable weight matrix, and σ is an activation function.

In traffic prediction, the goal is to capture both spatial patterns (graph structure) and temporal dynamics (time series). This led to the development of Spatio-Temporal GNNs, which combine GCNs with temporal models like RNNs or attention mechanisms to effectively model the complex dependencies in traffic networks.

3.3.1 Recurrent-Based Approaches

A prominent direction in ST-GNNs involves integrating graph convolution operations into recurrent neural network (RNN) cells. Seo et al. [6] introduced the Graph Convolutional Recurrent Network, which incorporates the graph convolution operator directly into the input-to-state and state-to-state transitions of LSTM and GRU cells. These models, referred to as **GConvLSTM** and **GConvGRU**, are capable of learning spatial structures that evolve over time. The key idea is to replace the standard matrix multiplication in RNNs with graph convolution operations, enabling the model to capture the topological relationships inherent in the road network. Mathematically, the state update for a GConvGRU can be expressed as:

$$z_t = \sigma(\text{GCN}(A, [x_t, h_{t-1}]; W_z)), \tag{9}$$

$$r_t = \sigma(\text{GCN}(A, [x_t, h_{t-1}]; W_r)), \tag{10}$$

$$\tilde{h}_t = \tanh(\text{GCN}(A, [x_t, r_t \odot h_{t-1}]; W_h)), \tag{11}$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t, \tag{12}$$

where h_t is the hidden state at time t, x_t is the input at time t, A is the adjacency matrix of the graph, z_t and r_t are the update and reset gates, $\tilde{h}_t$ is the candidate hidden state, W_z, W_r, W_h are learnable parameters of the graph convolutions, $\odot$ denotes the element-wise product, and σ is a non-linear activation function.

Similarly, Zhao et al. [7] proposed the Temporal Graph Convolutional Network (**TGCN**), which combines a Graph Convolutional Network (GCN) for spatial feature extraction with a GRU for temporal modeling. The GCN captures the topological relationships of the road network, while the GRU models the dynamic changes in traffic data over time. As a schematic single-layer summary, the spatial encoder can be written as

$$H^{(l+1)} = \sigma\left(\tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2}H^{(l)}W^{(l)} + b^{(l)}\right), \tag{13}$$

where $H^{(l)}$ represents the node embeddings at layer l, $W^{(l)}$ is the weight matrix, and $b^{(l)}$ is the bias term. The temporal dynamics are then modelled by integrating this spatial encoder into a gated recurrent cell. We adopt the formulation used in `torch_geometric_temporal.nn.recurrent.TGCN` [18], in which a separate graph convolution feeds each gate and the previous hidden state is concatenated before a linear projection:

$$z_t = \sigma\big(W_z\,[\,\mathrm{GCN}_z(x_t, A),\, h_{t-1}\,] + b_z\big), \tag{14}$$

$$r_t = \sigma\big(W_r\,[\,\mathrm{GCN}_r(x_t, A),\, h_{t-1}\,] + b_r\big), \tag{15}$$

$$\tilde{h}_t = \tanh\big(W_h\,[\,\mathrm{GCN}_h(x_t, A),\, r_t \odot h_{t-1}\,] + b_h\big), \tag{16}$$

$$h_t = z_t \odot h_{t-1} + (1 - z_t) \odot \tilde{h}_t, \tag{17}$$

where z_t and r_t are the update and reset gates, $\tilde{h}_t$ is the candidate hidden state, $[\cdot,\cdot]$ denotes column-wise concatenation, and $\odot$ denotes the element-wise product. Note that the update-gate convention here weights the *previous* hidden state by z_t, opposite to the Cho et al. GRU convention [12].

3.3.2 Attention-Based Approaches

While recurrent mechanisms are effective for capturing temporal dependencies, they may struggle with very long-term dependencies and can be computationally expensive. To address these limitations, attention mechanisms have been introduced [19]. Attention mechanisms allow the model to dynamically weigh the importance of different time steps in the historical data, enabling it to focus on the most relevant temporal information.

Zhu et al. [8] proposed the Attention Temporal Graph Convolutional Network (**A3TGCN**), which extends the capabilities of TGCN by incorporating an attention mechanism. This mechanism assigns varying importance to different time points, improving the model's ability to handle scenarios with high variability. The attention mechanism can be expressed as:

$$\alpha_t = \frac{\exp\left(\mathrm{score}(h_t, q)\right)}{\sum_{t'} \exp\left(\mathrm{score}(h_{t'}, q)\right)}, \tag{18}$$

where α_t is the attention weight for time step t, h_t is the hidden state at time t, q is the query vector, and score is a function that computes the relevance of h_t with respect to q. The final output is then computed as a weighted sum of the hidden states:

$$o = \sum_t \alpha_t h_t. \tag{19}$$

These advanced ST-GNN frameworks provide the theoretical foundation for the predictive model developed for the Bogotá traffic project, enabling the effective modeling of complex spatio-temporal dependencies in urban traffic networks.

4 Data Processing and Infrastructure

The deployment of ST-GNNs for traffic forecasting in Bogotá presented unique computational and engineering challenges that required specialized preprocessing and runtime contracts. The repository now implements a sparse graph-temporal pipeline that converts raw geospatial, traffic, event, incident, road-state, weather, and contextual inputs into HDF5 sparse matrices and graph metadata consumed by `TrafficDataset`. This section summarizes the processing path that supports both training and inference while preserving the graph consistency required by recurrent GNN models.

4.1 Feature Engineering and Data Sources

Our approach leverages a diverse array of urban data sources to capture the multifaceted nature of traffic dynamics in Bogotá. The feature engineering pipeline processes multiple heterogeneous datasets into a unified spatial-temporal representation suitable for graph-based learning.

4.1.1 Primary Traffic Features

The core traffic data is derived from Waze's crowdsourced traffic jams dataset, providing real-time insights into congestion patterns across the city's road network [2]. The `JamsProcessor` component extracts and spatially assigns traffic features to road segments through geometric matching. Traffic jams are mapped to specific road segments using a 4-meter buffer around each road's geometry, with overlap thresholds applied to refine the spatial join process.

To align the temporal granularity of the raw jam data with the model's requirements, the data was aggregated into 15-minute intervals. This temporal resolution was chosen to balance the need for capturing traffic dynamics with the computational efficiency required for large-scale data processing. The aggregated data includes multiple features for each road segment and time window, such as congestion severity (i.e., *jam level*), delay metrics, average speeds, and a count of severe congestion events (i.e., *jam count*). These features provide a multi-dimensional representation of traffic conditions, enabling the model to capture the complex dynamics of urban mobility.

Environmental and contextual factors, such as weather conditions, also play a significant role in influencing traffic patterns. To incorporate these factors, meteorological data from multiple weather stations was processed and integrated into the model. Key weather variables, including wind direction and speed, humidity, precipitation, and temperature, were collected and assigned to road segments based on their proximity to weather stations. A spatial buffer was used to ensure that each road segment received relevant environmental context. Furthermore, the weather data was temporally aligned with the traffic data using the same 15-minute resolution, ensuring consistency in the temporal representation of the dataset.

Static urban characteristics were also considered to provide additional context for traffic generation and flow patterns. Building footprints were extracted from OpenStreetMap and categorized by type, such as residential, commercial, industrial, and public buildings [20]. These classifications help capture the varying traffic generation patterns associated with different land uses. Building density metrics were calculated by counting the number of buildings within 100-meter buffers of each road segment, providing local density indicators. Additionally, population data from Transport Analysis Zones (ZAT, from the Spanish name *Zonas de Análisis de Transporte*) was spatially joined to road segments, offering demographic context for traffic demand modeling.

Disruptions to normal traffic flow, such as accidents, road closures, and public events, were also accounted for in the data processing pipeline. Traffic incidents were assigned directly to affected road segments using precise spatial matching techniques. Public events, such as concerts, demonstrations, and sporting events, were modeled using a network-based propagation approach. This method allowed the impact of events to spread through connected road segments based on the complexity and scale of the event, providing a more realistic representation of their influence on traffic patterns.

Finally, we implemented the `RoadGraphConstructor` component to transform Bogotá's road network into a directed graph. Each road segment was represented as a graph node, enriched with static features such as geometric properties (e.g., road length, cardinal direction, and flow directionality), land use context (e.g., building counts by type, population density, and land use composition), and transit infrastructure (e.g., bus stop counts and proximity to public transportation corridors). This comprehensive representation of the road network ensures that the model can effectively capture the unique characteristics of Bogotá's urban environment.

4.1.2 Edge Construction with Legal Traffic Flow

The construction of edges in the road network graph was carefully designed to incorporate the legal traffic flow constraints and spatial proximity relationships. Legal connections, which represent the navigable paths between road segments based on traffic directionality and intersection geometry, were assigned a weight of 1.0. These connections ensure that the graph accurately reflects the permissible movements within the road network, capturing the inherent structure and constraints of urban traffic.

In addition to legal connections, spatial proximity connections were introduced to account for the influence of geographically close but not directly connected road segments. These connections, assigned a lower weight of 0.3, allow the ST-GNN to learn spatial patterns that may exist between nearby road segments, even in the absence of direct connectivity. This approach enhances the model's ability to capture the complex spatial dependencies present in urban traffic networks while maintaining respect for the legal traffic flow constraints.

4.2 Memory-Efficient Data Storage and Retrieval

The scale and sparsity of urban traffic data posed significant challenges for storage and retrieval. To address these challenges, a memory-efficient data storage and retrieval system was developed, leveraging the Coordinate (COO) sparse matrix format [21, 22]. This format is particularly well-

suited for traffic data, which is characterized by a high degree of sparsity, as most road segments do not experience traffic incidents or specific events at any given time. By storing only non-zero feature values, the COO format significantly reduces the memory footprint.

Each data point in the COO format is represented as a tuple containing the node index, time index, feature index, and value. This structure enables rapid retrieval of specific spatiotemporal slices, facilitating efficient data access during model training. Furthermore, the data is sorted by time index, which allows for the use of binary search operations to perform time-range queries efficiently.

For example, consider the following dense matrix representation of traffic data:

	t_1	t_2	t_3
n_1	0	5	0
n_2	0	0	3
n_3	7	0	0

In the COO format, only the non-zero entries are stored along with their corresponding indices:

Node Index	Time Index	Value
n_1	t_2	5
n_2	t_3	3
n_3	t_1	7

As shown, the COO representation significantly reduces the storage requirements by omitting zero entries, making it an ideal choice for handling sparse traffic data.

To further optimize storage and retrieval, the processed sparse data is stored in the HDF5 format. This format offers several advantages, including lazy loading, which ensures that only the required data slices are loaded into memory during training [23]. This capability is particularly important for processing datasets that exceed the available RAM, as it enables efficient training without the need for extensive hardware resources. Additionally, HDF5's built-in compression reduces disk storage requirements while maintaining rapid access times. Pre-computed time-based indices are also cached, enabling efficient access to temporal windows during training.

These innovations in data storage and retrieval were critical for managing the scale and complexity of the urban traffic data. The combination of sparse matrix representation and HDF5 storage architecture ensures that the training process remains computationally feasible while preserving the rich spatiotemporal relationships essential for accurate traffic forecasting.

4.3 `TrafficDataset` Runtime Contract

The `TrafficDataset` contract is the central runtime abstraction that connects the processed sparse data to the ST-GNN training loop. The dataset keeps graph metadata and static node features in memory, while dynamic features and targets are retrieved lazily for the requested temporal window. This design avoids loading the full dense city-time-feature tensor, which would be unnecessarily large given the sparsity of traffic incidents, irregularities, events, and several contextual sources.

The most important constraint imposed by this dataset contract is graph consistency inside a batch. A training sample is not defined only by a timestamp; it is defined by a seed-local subgraph and a time window. All examples in a batch must therefore share one compatible `edge_index`. The current implementation preserves this invariant by allowing the time index to vary inside a batch while keeping the seed fixed. In practical terms, each batch contains one seed-local road subnetwork and multiple historical windows drawn from that same subnetwork.

This contract also determines how missing sparse values are interpreted. For traffic severity scoring and dense reconstruction, missing sparse entries represent the absence of an observed congestion signal and are treated as level 0. This convention is especially important for dense real-time evaluation, because failing to count missing traffic observations as zero would inflate the apparent prevalence of high-severity samples.

4.4 Missing Data Imputation Strategy

Urban traffic data often contains missing values due to factors such as sensor failures, data transmission issues, and inconsistent data source coverage. To address this challenge, a neighbour-based imputation strategy is implemented in `src/preprocessing/imputation.py`. For each road segment, the procedure first computes a per-segment time-average of the observed irregularity features *trend* and *regularSpeed*. It then walks the road graph and, for every node, extracts its k-hop ego subgraph (default $k = 10$ hops, where hops are graph edges rather than metres). Within this neighbourhood, only nodes that (i) have an observed mean for the feature and (ii) share the same street type (`MVITIPO`) as the target node are retained, and the imputed value is the unweighted arithmetic mean of their per-segment averages [**?**].

When this graph-based step yields no valid same-type neighbours within k hops, the segment falls back to the global mean of the feature for its street type. Nodes that lie outside the largest connected component of the road graph bypass the neighbourhood step entirely and are imputed directly from the same street-type fallback. This two-stage fallback (same-street-type neighbours $\rightarrow$ same-street-type global mean) guarantees that every segment receives a value while keeping the imputation consistent with the local road typology.

4.5 Normalization and Scale Handling

The diverse range of features in the dataset, which included binary event indicators, continuous speed measurements, and other heterogeneous data types, necessitated a robust normalization strategy to ensure numerical stability and preserve the relative importance of each feature. To address this, feature-wise normalization was applied, with each feature being independently normalized using dataset-specific statistics. This approach ensured that all features were scaled appropriately, facilitating effective learning by the model.

To maintain consistency between the training and inference phases, normalization constants were computed globally across the entire temporal range of the dataset. This ensured that the normalized values were comparable across different time periods, avoiding potential discrepancies that could arise from using separate normalization parameters for training and inference. To minimize memory overhead, normalization was applied on-the-fly during data loading using pre-computed tensors. This approach allowed the system to handle large datasets efficiently while

maintaining the integrity of the normalized data.

By implementing these strategies for sparse storage, seed-time batching, missing data imputation, and normalization, the data processing pipeline prepares the urban traffic data for training sophisticated ST-GNN models. This comprehensive data processing pipeline enables the training of these complex models on city-scale traffic data while maintaining computational efficiency and preserving the rich spatial-temporal relationships essential for accurate traffic forecasting.

5 Seed-Time Sampling Methodology

One of the main empirical challenges in the project is the imbalance of congestion states. Urban traffic data contains many road segments and time windows with no meaningful congestion signal, while the operationally important cases are precisely the medium- and high-severity events that occur less frequently. Earlier training runs were therefore dominated by low-severity samples [24]. A sampler that merely walks through global time indices can improve temporal coverage, but it does not fully align sample severity with the actual seed-local graph seen by the model in each batch.

5.1 Sampling Unit

The current methodology defines the durable sampling unit as `(seed_id, time_idx)` rather than `time_idx` alone. This distinction is central to the training design. The model observes a sample through a seed-local road subgraph, so a time window that appears severe globally may not contain meaningful congestion inside a particular seed-local neighborhood. Conversely, a localized congestion event may be important for one subgraph even if the global city-level average is modest. By scoring seed-time pairs directly, the sampler can select examples that are informative for the graph actually presented to the model.

5.2 Artifact Construction

The seed-time artifact is precomputed outside the training loop to avoid expensive severity scoring during model training. The artifact contains rows keyed by seed and time index, with a congestion score and a bucket label. The final combined universe currently covers 1,937 seeds and 2,000 selected time indices, yielding 3,874,000 seed-time rows. The resulting bucket distribution is sufficiently populated in the medium and high buckets to support stratified oversampling, despite the dominance of the low bucket [25]. This is enough for controlled sampling: 1,939,990 low rows, 963,980 medium rows, and 970,030 high rows. The thresholding procedure uses the corrected convention that sparse missing target values count as zero, which prevents overestimating the frequency of severe congestion.

5.3 Epoch Planning

During training, the artifact is not sampled row by row as if every seed-time pair were independent. Instead, the runtime preserves the `TrafficDataset` graph constraint by planning one seed per batch and multiple time indices within that seed. In terms of implementation, when

the parameter `seed-time-artifact-path` is provided, `training_entrypoint.py` downloads and caches the artifact if necessary, `SeedTimeEpochPlanner` groups rows by seed and bucket, and `PlannedSeedTimeBatchSampler` maps the resulting plan back into standard dataset indices.

With a batch size of 16 and one epoch cycle, the intended combined-artifact plan yields 1,937 batches per epoch and 30,992 sample slots. The target bucket mix is approximately 44% high, 31% medium, and 25% low congestion. In the practical integer allocation for a 16-window batch, this corresponds to 7 high, 5 medium, and 4 low samples. This choice deliberately increases exposure to informative congestion cases while retaining enough low-congestion examples to prevent the model from losing calibration on normal traffic conditions.

6 Model Architecture and Training Infrastructure

The implementation of ST-GNN for Bogotá's traffic forecasting system required careful consideration of model architecture design, scalable training infrastructure, and optimization strategies tailored to the unique challenges of urban traffic prediction.

6.1 Unified ST-GNN Architecture Framework

Our system implements a unified framework supporting multiple state-of-the-art ST-GNN architectures, allowing for empirical comparison and selection of the most suitable model for Bogotá's traffic patterns.

6.1.1 Model Architecture Components

The proposed framework is composed of three integral components that collectively enable the processing of multi-modal urban data. The first component is the static feature encoder, which utilizes a Multilayer Perceptron (MLP) to transform static road features, such as geometry, land use, and population data, into dense embeddings. This encoder maps high-dimensional static features into a 32-dimensional representation, ensuring that each road segment is enriched with consistent geometric and demographic context.

The second component is the temporal graph convolution module, which serves as the core of the ST-GNN. This module processes historical traffic patterns by employing graph-aware recurrent cells. By concatenating dynamic features with the static embeddings generated by the first component, the temporal graph convolution module is able to process temporal sequences while preserving the spatial relationships inherent in the road network graph. This enables the model to capture the complex spatio-temporal dependencies that characterize urban traffic dynamics.

The third component is the future feature integration module, which is designed to incorporate known future features, such as planned events and weather forecasts, into the prediction process. This module employs a separate recurrent architecture to process these external factors over the forecast horizon, thereby enhancing the model's ability to predict future traffic conditions with greater accuracy.

6.1.2 Supported ST-GNN Variants

The framework is designed to support multiple ST-GNN architectures, while the latest completed comparison line focuses on three active model families. The first variant, GConvGRU, integrates Graph Convolutional Networks (GCNs) with Gated Recurrent Units (GRUs). This architecture is particularly well-suited for capturing short to medium-term traffic dependencies due to its efficient memory usage and rapid training convergence.

The second variant, Temporal Graph Convolutional Network (TGCN), combines GCN-based spatial processing with GRU-based temporal modeling in a streamlined architecture. TGCN strikes an effective balance between computational efficiency and predictive accuracy, making it a robust choice for modeling urban traffic patterns.

The third variant, GConvLSTM, extends the capabilities of GCNs by incorporating Long Short-Term Memory (LSTM) cells. This architecture excels in modeling longer-term dependencies, such as weekly and seasonal traffic patterns, although it comes with increased computational complexity.

A fourth variant, Attention Temporal Graph Convolutional Network (A3TGCN), remains a supported reference architecture. It enhances the TGCN architecture by introducing temporal attention mechanisms that dynamically assign weights to historical time steps. However, A3TGCN is not currently part of the active comparison line, as the focus has been on evaluating the performance of GConvGRU, TGCN, and GConvLSTM variants.

6.1.3 Multi-Target Prediction Architecture

A distinguishing feature of the proposed framework is its capability for multi-target prediction, which allows for the simultaneous forecasting of multiple traffic metrics. The primary target of the model is the jam level, which quantifies congestion severity on a 6-point integer scale (0 for no congestion, 5 for severe congestion) and serves as the core variable for assessing traffic conditions. In addition to the jam level, the model also predicts the mean jam velocity, which provides complementary insights into the characteristics of traffic flow.

The framework employs a joint optimization strategy to predict both targets simultaneously. By leveraging a shared representation, the model is able to learn correlated patterns between the jam level and mean jam velocity, thereby improving the robustness and accuracy of its predictions [26]. This multi-target approach ensures that the model captures a more comprehensive understanding of traffic dynamics, which is critical for effective traffic management and planning in complex urban environments.

6.2 Scalable Training Infrastructure

Training ST-GNN models on city-scale data requires sophisticated infrastructure capable of handling the computational and memory demands of large graph neural networks.

6.2.1 AWS SageMaker Integration

The training infrastructure for the ST-GNN models is built on AWS SageMaker, which provides a scalable and managed environment for training. The canonical runtime entrypoint is

`src/training_entrypoint.py`, typically launched through the SageMaker training notebook so that the model family, image, VPC configuration, EFS mount, S3 output path, and seed-time artifact parameters are recorded consistently. Recent runs used GPU-optimized SageMaker instances including `ml.g4dn.8xlarge`, `ml.g5.4xlarge`, and `ml.g5.8xlarge`, depending on the model family.

To ensure reproducibility and consistency across different infrastructure configurations, the training environment is containerized using custom Docker containers. These containers include all necessary dependencies, such as PyTorch [27], PyTorch Geometric [28], and other specialized graph neural network libraries. This approach simplifies the deployment process and ensures that the training environment remains consistent across different runs and setups.

Additionally, the training pipeline integrates with the AWS Elastic File System (EFS), which provides high-throughput and low-latency access to preprocessed datasets. Seed-time artifacts and final model outputs are exchanged through S3, while the entrypoint can download and cache the sampling artifact locally for the training job. This design separates large preprocessed data, sampling metadata, and trained model artifacts while keeping the runtime reproducible enough for model-family comparisons.

6.2.2 Advanced Memory Management

Given the memory-intensive nature of large-scale graph processing, the training system incorporates several advanced memory optimization techniques. One such technique is dynamic graph batching, where edge indices are constructed dynamically for each batch. This approach allows the creation of batched graph representations that maximize GPU utilization while adhering to memory constraints.

Another critical optimization is gradient accumulation, which enables the use of large effective batch sizes by accumulating gradients across multiple mini-batches. This technique is particularly useful for achieving stable training when GPU memory is limited [29]. Furthermore, the system employs Automatic Mixed Precision (AMP) training with GradScaler. This method reduces memory usage by performing computations in lower precision while maintaining numerical stability through dynamic loss scaling [30]. These memory management strategies are essential for ensuring that the training process remains efficient and scalable, even when dealing with the extensive data and computational requirements of urban traffic prediction.

6.3 Training Strategy and Optimization

The training process for the ST-GNN models is designed to address the unique challenges of urban traffic prediction. A key aspect of the training strategy is the use of temporal data splitting. Unlike traditional random data splitting, which can lead to unrealistic evaluation scenarios, this approach employs a chronological division of the dataset. Specifically, the training data comprises the first 80% of the temporal sequence, while the most recent 20% is reserved for validation. This method ensures that the evaluation process mimics real-world deployment scenarios, where future data is not available during model training. By strictly preventing future information from influencing past predictions, the temporal splitting approach ensures that the model's performance is evaluated in a realistic and unbiased manner.

The loss function used for training the ST-GNN models is a composite objective that can be written in compact form as

$$L \;=\; L_{\text{base}} \;+\; \lambda_s L_{\text{shape}}, \tag{20}$$

where L_{base} aggregates several termwise penalties that operate on finite target entries, and L_{shape} is an auxiliary node-wise Pearson shape loss applied over the forecast horizon. Below we give a precise definition of each term used in the implementation (see `src/training/losses.py`).

Let p be the model prediction tensor and y the target tensor (both shaped so that the last dimension indexes the forecast horizon). Define the finite-entry mask

$$M = \mathbf{1}_{\text{finite}}(y), \tag{21}$$

and the scaled real-valued predictions and targets

$$\hat{p} = s\,p, \qquad \hat{y} = s\,y, \tag{22}$$

where s is the scalar level scale (named `level_scale` in code).

Let the rounded target class be

$$\ell(y) = \text{clamp}(\text{round}(\hat{y}), 0, 5) \in \{0, \ldots, 5\}. \tag{23}$$

Let b_c for $c = 0, \ldots, 5$ denote the base level weights (default $[0.25, 1, 2, 3, 4, 5]$). The per-element weight used across most terms is

$$w(i) = b_{\ell(y_i)}\, M_i, \tag{24}$$

where index i enumerates all scalar prediction locations. To avoid divide-by-zero, all denominators are clamped to a small positive floor; the implementation uses a floor of 1.0 for the terms in L_{base} and a small ε in the shape loss.

Continuous regression term. Let $\delta_i = p_i - y_i$ be the error on the *unscaled* predictions and targets. Depending on the configuration this term is one of:

$$\text{MAE: } h(\delta_i) = |\delta_i|, \tag{25}$$

$$\text{MSE: } h(\delta_i) = \delta_i^2, \tag{26}$$

$$\text{Huber}(\Delta)\ [31]\text{: } h(\delta_i) = \begin{cases} \frac{1}{2}\delta_i^2 & |\delta_i| \le \Delta, \\ \Delta(|\delta_i| - \frac{1}{2}\Delta) & |\delta_i| > \Delta\ . \end{cases} \tag{27}$$

The continuous loss is the weight-normalized sum

$$L_{\text{cont}} = \frac{\sum_i w(i)\, h(\delta_i)}{\sum_i w(i)}. \tag{28}$$

Differentiable class cross-entropy. For each scalar forecast location we form class logits by placing a Gaussian-shaped (squared-distance) logit around each integer class center $c \in \{0, \ldots, 5\}$:

$$z_{i,c} = -\big(\hat{p}_i - c\big)^2. \tag{29}$$

The cross-entropy loss is applied to these logits on masked entries and averaged with the same per-element weight:

$$L_{\mathrm{CE}} = \frac{\sum_{i:\, M_i=1} w(i)\, \mathrm{CE}(z_{i,\cdot},\, \ell(y_i))}{\sum_{i:\, M_i=1} w(i)}. \tag{30}$$

Ordinal distance penalties. Let the predicted rounded class be $\hat{\ell}(p) = \mathrm{clamp}(\mathrm{round}(\hat{p}), 0, 5)$ and define the absolute class distance $d_i = |\hat{\ell}(p_i) - \ell(y_i)|$. The basic ordinal penalty is

$$L_{\mathrm{dist}} = \frac{\sum_i w(i)\, d_i}{\sum_i w(i)}. \tag{31}$$

An additional squared-distance penalty is applied to large class errors (indices with $d_i \geq T_{\mathrm{large}}$):

$$L_{\mathrm{large}} = \frac{\sum_{i:\, d_i \geq T_{\mathrm{large}}} w(i)\, d_i^2}{\sum_{i:\, d_i \geq T_{\mathrm{large}}} w(i)}. \tag{32}$$

High- and severe-level MAE terms. Restricting to locations with target class above a threshold T_{high} yields a weighted MAE on the unscaled errors δ_i:

$$L_{\mathrm{high}} = \frac{\sum_{i:\, \ell(y_i) \geq T_{\mathrm{high}}} w(i)\, |\delta_i|}{\sum_{i:\, \ell(y_i) \geq T_{\mathrm{high}}} w(i)}. \tag{33}$$

Additionally, a piecewise severe term penalizes absolute errors at locations where the scaled target exceeds a severe threshold T_{severe}:

$$L_{\mathrm{piece}} = \frac{\sum_{i:\, \hat{y}_i \geq T_{\mathrm{severe}}} w(i)\, |\delta_i|}{\sum_{i:\, \hat{y}_i \geq T_{\mathrm{severe}}} w(i)}. \tag{34}$$

Directional underprediction penalty. This term operates on the unscaled predictions and targets, restricted to finite target locations. For indices where $M_i = 1$ and $p_i < y_i$:

$$L_{\mathrm{under}} = \frac{\sum_{i:\, M_i=1,\, p_i<y_i} w(i)\, (y_i - p_i)}{\sum_{i:\, M_i=1,\, p_i<y_i} w(i)}. \tag{35}$$

Aggregate base objective. Using scalar coefficients (whose implementation names are listed below) we combine the above:

$$L_{\mathrm{base}} = L_{\mathrm{cont}} + \lambda_{\mathrm{CE}} L_{\mathrm{CE}} + \lambda_{\mathrm{dist}} L_{\mathrm{dist}} + \lambda_{\mathrm{large}} L_{\mathrm{large}} + \lambda_{\mathrm{high}} L_{\mathrm{high}} + \lambda_{\mathrm{piece}} L_{\mathrm{piece}} + \lambda_{\mathrm{under}} L_{\mathrm{under}}. \tag{36}$$

The coefficients correspond to `class_ce_lambda` (λ_{CE}), `class_distance_lambda` (λ_{dist}), `large_error_lambda` (λ_{large}), `high_level_lambda` (λ_{high}), `piecewise_mae_lambda` (λ_{piece}), `directional_lambda` (λ_{under}), and `shape_lambda` (λ_s). The thresholds correspond to `large_error_threshold` (T_{large}), `high_level_threshold` (T_{high}), and `piecewise_threshold` (T_{severe}).

Node-wise Pearson shape loss. For each node-series (all non-horizon dimensions collapsed into an index n) consider the horizon entries $t = 1, \ldots, H$ and restrict to those t where both $\hat{p}_{n,t}$ and $\hat{y}_{n,t}$ are finite. Denote the finite count N_n, and let $\bar{p}_n$, $\bar{y}_n$ be the finite-entry means. Define

the population covariance and standard deviations

$$\mathrm{cov}_n = \frac{1}{N_n} \sum_{t \in \mathcal{T}_n} (\hat{p}_{n,t} - \bar{p}_n)(\hat{y}_{n,t} - \bar{y}_n), \qquad \sigma_{p,n} = \sqrt{\Big(\tfrac{1}{N_n} \sum_{t \in \mathcal{T}_n} (\hat{p}_{n,t} - \bar{p}_n)^2 \Big) + \varepsilon}, \tag{37}$$

and $\sigma_{y,n}$ analogously, where $\mathcal{T}_n$ denotes the set of horizon indices for which both $\hat{p}_{n,t}$ and $\hat{y}_{n,t}$ are finite. The Pearson correlation for node n is

$$\rho_n = \mathrm{clamp}\Big(\frac{\mathrm{cov}_n}{\sigma_{p,n}\, \sigma_{y,n} + \varepsilon},\, -1,\, 1 \Big). \tag{38}$$

The node weight combines the population standard deviation of the target with its range and maximum. Define the zero-filled target series $\tilde{y}_{n,t} = \hat{y}_{n,t}\, \mathbf{1}_{t \in \mathcal{T}_n}$ (i.e. non-finite entries are replaced by zero). The node weight is then

$$\omega_n = 1 + \alpha_{\mathrm{std}}\, \sigma_{y,n} + \alpha_{\mathrm{range}}\, \max\Big(0,\, \max_t \tilde{y}_{n,t} - \min_t \tilde{y}_{n,t}\Big) + \alpha_{\max}\, \max\Big(0,\, \max_t \tilde{y}_{n,t}\Big). \tag{39}$$

Only nodes with $N_n \geq 2$ and $\sigma_{y,n} > \mathrm{min_std}$ are included. The weighted shape loss is

$$L_{\mathrm{shape}} = \frac{\sum_{n \in \mathcal{V}} \omega_n\, (1 - \rho_n)}{\sum_{n \in \mathcal{V}} \omega_n}, \tag{40}$$

where $\mathcal{V}$ is the set of valid nodes.

Putting it all together yields the final training objective used in code. All termwise sums are implemented with safe masking and small-clamp denominators to ensure numerical stability; hyperparameters in the implementation map directly to the coefficients and thresholds above (for exact names and defaults see `src/training/losses.py`).

To ensure comprehensive and informative coverage during training, the current runtime uses the seed-time planner described above when a sampling artifact is provided. The legacy seed-based sampler remains available, but the active direction is artifact-driven planning over seed and time pairs with controlled congestion-bucket ratios.

@inproceedingspascanu2013difficulty, The training configuration is designed to balance computational efficiency with model performance. Input sequences of 150 timesteps (equivalent to 37.5 hours) are used, with output horizons of 50 timesteps (12.5 hours). This configuration provides sufficient historical context for the model to learn from while maintaining practical forecast utility. The training process employs the Adam optimizer [32] with a learning rate of 0.001, a gradient clipping [33] norm of 1.0, and a batch size of 16, with each batch containing approximately 500 nodes. To minimize I/O bottlenecks during GPU computation, the data loading process is optimized using multi-worker data loading with four workers, persistent workers, and prefetching with a factor of two. Training stability measures, including tensor validation, NaN/infinity detection, and statistical monitoring, are implemented to prevent training instabilities and ensure the robustness of the model.

The training system also incorporates monitoring and validation capabilities to ensure the quality and reliability of the trained models. Runtime logs include loss metrics, congestion-level operational metrics, and timing metrics such as data wait, transfer, forward pass, loss computation, backward pass, optimizer step, and total step time. Integration with AWS SageMaker's

metric definitions allows automatic capture and visualization through AWS CloudWatch, although the parsed metric surface is intentionally compressed to remain within SageMaker metric-definition limits. Regular model checkpointing is performed to preserve the model state, enabling model selection based on validation performance.

By combining advanced memory management techniques, a robust training strategy, and comprehensive monitoring capabilities, the training infrastructure is well-equipped to handle the challenges of training sophisticated ST-GNN models on large-scale urban traffic data. This ensures that the resulting models are both accurate and efficient, making them suitable for deployment in real-world traffic management systems.

7 Experimental Setup

The current experimental setup is designed to approximate operational forecasting while avoiding temporal leakage. The processed time axis is split chronologically, with earlier windows used for training and later windows reserved for validation. This is preferable to random splitting because real deployment always predicts future conditions from past observations. The model receives 150 historical timesteps, equivalent to 37.5 hours at a 15-minute resolution, and predicts 50 future timesteps, equivalent to 12.5 hours.

The active comparison line focuses on GConvLSTM, GConvGRU, and TGCN. Training jobs are launched through SageMaker using the current container image, the EFS-mounted processed dataset, model-family-specific S3 output prefixes, and the final seed-time artifact when the planner-based sampler is enabled. The latest completed comparison line corresponds to the 2026-04-20 shape-loss experiments. Those runs use the newer sampling artifact and the auxiliary shape-following loss described above.

The evaluation metrics are intentionally divided into two families. Offline validation metrics measure performance on the historical validation split produced from the sparse training dataset. These include exact rounded-level accuracy, plus-or-minus-one rounded-level accuracy, level MAE, and per-level behavior when available. Dense real-time evaluation measures the behavior of production-style prediction surfaces against Waze jam observations, after interpreting missing jam observations as level 0. These two evaluation surfaces are complementary, but they should not be treated as numerically interchangeable because they evaluate different data distributions.

Table 1: Current training and evaluation configuration.

Component	**Current configuration**
Historical input horizon	150 timesteps, 37.5 hours
Forecast horizon	50 timesteps, 12.5 hours
Temporal split	Chronological train-validation split
Batch structure	One seed-local graph, multiple time windows
Batch size	16 time windows per seed batch
Active model families	GConvLSTM, GConvGRU, TGCN
Sampling artifact	1,937 seeds and 3,874,000 seed-time rows
Primary runtime	AWS SageMaker with EFS and S3 artifacts

8 Results and Current Evaluation Status

8.1 Offline Validation Results

The latest completed SageMaker comparison line uses the 2026-04-20 shape-loss runs for GConvLSTM, GConvGRU, and TGCN. In this validation line, GConvLSTM is the strongest global model, with validation exact accuracy of 0.6538, plus-or-minus-one accuracy of 0.8357, and level MAE of 0.6301. GConvGRU remains strongest on high-congestion levels 3 and 4 among the latest three-model comparison, while TGCN shows the largest improvement relative to its previous version, driven primarily by lower congestion levels.

Table 2: Current offline validation reading from the latest completed comparison line.

Model family	Current reading	Interpretation
GConvLSTM	Best global validation metrics	Exact accuracy: 0.6538 ± 1 accuracy: 0.8357 Level MAE: 0.6301
GConvGRU	Strongest on levels 3 and 4	Most relevant current candidate for high-congestion behavior
TGCN	Largest improvement vs. previous line	Improvement appears driven mainly by lower congestion levels

These results suggest that the model-family decision is not a simple global-metric ranking problem. GConvLSTM is the most attractive model when the objective is aggregate validation performance, but GConvGRU remains important because the operational value of the system depends heavily on behavior during non-trivial congestion. TGCN is useful as a compact comparison point and as evidence that the newer training line can materially change model behavior, even when improvements are concentrated in lower levels.

8.2 Dense Real-Time Evaluation

Real-time evaluation is now active against dense prediction surfaces produced by the sibling inference system. Dense evaluation interprets missing Waze jams as level 0 and compares the full prediction surface rather than only observed jam records. A stable partial summary over 10 prediction files reports MAE of 1.7331, RMSE of 1.9473, bias of 1.6535, exact accuracy of 0.1897, and plus-or-minus-one accuracy of 0.3351.

Table 3: Partial dense real-time evaluation over 10 prediction files.

Metric	Value
MAE	1.7331
RMSE	1.9473
Bias	1.6535
Exact accuracy	0.1897
± 1 accuracy	0.3351

These values are not directly comparable to sparse validation metrics because they evaluate a denser and more operationally honest surface. The current pattern is that low congestion levels

are overpredicted, levels 3 and 4 are underpredicted, and the model nevertheless shows useful signal in levels 2 and 3 under dense evaluation. This is an important distinction: the system has learned meaningful congestion structure, but the current deployed prediction surface still requires calibration and stronger validation before it can support final claims about operational accuracy.

8.3 Interpretation

The combined evidence points to a system that is technically mature enough for structured experimentation but not yet complete enough for final deployment-grade reporting. The sparse validation results show that the ST-GNN training pipeline can learn the historical target distribution with useful accuracy. The dense real-time results reveal a harder operational reality, in which the model must make predictions over many road-time pairs where explicit jam observations are absent. Under that denser evaluation surface, the model currently tends to overpredict low levels while still missing some severe events.

This gap between offline and dense evaluation is not a contradiction; it is a diagnostic result. It indicates that the next stage of work should focus on calibration, per-level behavior, and traceable inference artifacts rather than only on reducing aggregate training loss. In particular, the high-congestion behavior of GConvGRU, the global strength of GConvLSTM, and the improvement pattern of TGCN should be compared using a completed dense evaluation window before selecting a final operational model.

9 Deployment, Traceability, and Operational Constraints

The training repository and the production inference repository share core modeling concepts but are not identical codebases. The training repository is responsible for preprocessing, sparse historical datasets, SageMaker training, model comparison, and offline evaluation. The inference repository is responsible for real-time data ingestion, Redis integration, GPU-oriented seed batching, prediction reconstruction, and storage of prediction outputs. This separation is appropriate because training and serving have different performance and data-availability constraints, but it also creates explicit parity responsibilities.

The most important operational distinction is the difference between Redis prediction outputs and S3 prediction backups. The main Redis prediction key is filtered to emphasize predictions with meaningful congestion levels, while the S3 backup is intended to contain the full unfiltered prediction surface. For dense retrospective evaluation, the S3 backup is therefore the correct source, because evaluating only filtered Redis outputs would bias the results toward higher predicted congestion.

Traceability remains a practical limitation. The inference runtime constructs metadata such as model file, model hash, model version, and model configuration in memory, and some of this metadata is persisted to Redis payloads. However, S3 parquet backups do not currently persist durable model identity metadata. In addition, training checkpoints do not yet store a stable run identity such as run id, SageMaker training job name, model artifact source, or explicit version. This means that retrospective evaluation can be technically correct at the prediction-file level

while still being weaker than desired for model lineage and auditability.

10 Limitations and Next Steps

Several limitations remain before the paper can present final deployment-grade claims. The last-seven-day dense evaluation is still incomplete, so the current dense metrics should be interpreted as a stable partial snapshot rather than as final production evidence. The new seed-time planner and sampler also need more targeted unit tests and richer observability, including realized bucket counts per epoch, explicit plan size, and explicit shape-loss component metrics.

The next experimental priority is to complete the larger dense evaluation over the full last-seven-day prediction set and summarize metrics globally, by true congestion level, and by forecast step. This should be followed by a calibration decision: either recalibrate the current best model line, retrain with further loss and sampling adjustments, or pursue both paths in parallel. Because the current dense evaluation shows overprediction at low levels and underprediction at high levels, calibration should be evaluated not only by global MAE but also by false positives for level 0 and recall-like behavior for levels 3 and 4.

The next engineering priority is to improve traceability. S3 prediction backups should persist model identity fields such as `model_version`, `model_md5`, `training_job_name`, `model_artifact_s3`, and inference code version. Training checkpoints should likewise save durable identity fields so that the inference system can resolve a model version without relying on informal artifact names. These changes would make future evaluation tables more defensible because every prediction file could be linked to a specific training run and model artifact.

11 Conclusion

This white paper has presented an end-to-end Spatio-Temporal Graph Neural Network forecasting system for Bogotá's road network, jointly contributed by the Government of Bogotá and the Inter-American Development Bank. Beyond the model families themselves (GConvLSTM, GConvGRU, and TGCN), the work delivers a reusable engineering substrate: a sparse HDF5 graph-temporal data contract, a seed-time sampler that explicitly rebalances exposure across congestion buckets, and an evaluation path that separates historical sparse validation from dense operational behaviour.

The current comparison line shows that the pipeline can train all three architectures to useful sparse-validation accuracy, with GConvLSTM strongest on aggregate metrics, GConvGRU strongest on high-congestion levels, and TGCN producing the largest relative improvement over its previous baseline. The dense real-time evaluation surface, however, is not yet numerically comparable to the sparse validation results and currently exhibits systematic overprediction at low levels and underprediction at high levels.

Before a deployment-grade reading can be reported, two gating steps remain: completing the full last-seven-day dense evaluation summarised globally, by true congestion level, and by forecast step; and closing the traceability gap by persisting durable model identity in both training checkpoints and S3 prediction backups. Together these steps will let a future iteration

of this paper present model-family selection as a defensible, fully reproducible decision rather than a stable partial reading.

References

[1] P. Akbar, V. Couture, G. Duranton, and A. Storeygard, "The fast, the slow, and the congested: Urban transportation in rich and poor countries," 2023.

[2] F. Bedoya-Maya, A. Calatayud, and V. González Mejia, "Estimating the effect of urban road congestion on air quality in latin america," 2022.

[3] J. I. Levy, J. J. Buonocore, and K. von Stackelberg, "Evaluation of the public health impacts of traffic congestion: a health risk assessment," *Environmental Health: A Global Access Science Source*, vol. 9, p. 65, 2010.

[4] J. Chen, G. Yang, Z. Zhang, and W. Wang, "St-d3ddarn: Urban traffic flow prediction based on spatio-temporal decoupled 3d densenet with attention resnet," *PLOS ONE*, vol. 19, pp. 1–31, 06 2024.

[5] C. Zhang, Y. Wu, Y. Shen, S. Wang, X. Zhu, and W. Shen, "Adaptive graph convolutional recurrent network with transformer and whale optimization algorithm for traffic flow prediction," *Mathematics*, vol. 12, no. 10, 2024.

[6] Y. Seo, M. Defferrard, P. Vandergheynst, and X. Bresson, "Structured sequence modeling with graph convolutional recurrent networks," 2016.

[7] L. Zhao, Y. Song, C. Zhang, Y. Liu, P. Wang, T. Lin, M. Deng, and H. Li, "T-gcn: A temporal graph convolutional network for traffic prediction," *IEEE Transactions on Intelligent Transportation Systems*, vol. 21, pp. 3848–3858, Sept. 2020.

[8] J. Zhu, Y. Song, L. Zhao, and H. Li, "A3t-gcn: Attention temporal graph convolutional network for traffic forecasting," 2020.

[9] B. Williams and L. Hoel, "Modeling and forecasting vehicular traffic flow as a seasonal arima process: Theoretical basis and empirical results," *Journal of Transportation Engineering*, vol. 129, pp. 664–672, 11 2003.

[10] Y. Lv, Y. Duan, W. Kang, Z. Li, and F.-Y. Wang, "Traffic flow prediction with big data: A deep learning approach," *Ieee transactions on intelligent transportation systems*, vol. 16, no. 2, pp. 865–873, 2014.

[11] S. Hochreiter, "Long short-term memory," *Neural Computation MIT-Press*, 1997.

[12] K. Cho, B. Van Merriënboer, Ç. Gulçehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning phrase representations using rnn encoder–decoder for statistical machine translation," in *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*, pp. 1724–1734, 2014.

[13] H. Yao, F. Wu, J. Ke, X. Tang, Y. Jia, S. Lu, P. Gong, J. Ye, and Z. Li, "Deep multi-view spatial-temporal network for taxi demand prediction," 2018.

[14] M. M. Bronstein, J. Bruna, Y. LeCun, A. Szlam, and P. Vandergheynst, "Geometric deep learning: going beyond euclidean data," *IEEE Signal Processing Magazine*, vol. 34, no. 4, pp. 18–42, 2017.

[15] F. Scarselli, M. Gori, A. C. Tsoi, M. Hagenbuchner, and G. Monfardini, "The graph neural network model," *IEEE transactions on neural networks*, vol. 20, no. 1, pp. 61–80, 2008.

[16] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," 2017.

[17] M. Defferrard, X. Bresson, and P. Vandergheynst, "Convolutional neural networks on graphs with fast localized spectral filtering," *Advances in neural information processing systems*, vol. 29, 2016.

[18] B. Rozemberczki, P. Scherer, Y. He, G. Panagopoulos, A. Riedel, M. Astefanoaei, O. Kiss, F. Beres, G. Lopez, N. Collignon, *et al.*, "Pytorch geometric temporal: Spatiotemporal signal processing with neural machine learning models," in *Proceedings of the 30th ACM international conference on information & knowledge management*, pp. 4564–4573, 2021.

[19] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," *arXiv preprint arXiv:1409.0473*, 2014.

[20] M. Haklay and P. Weber, "Openstreetmap: User-generated street maps," *IEEE Pervasive computing*, vol. 7, no. 4, pp. 12–18, 2008.

[21] P. Virtanen, R. Gommers, T. E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, E. Burovski, P. Peterson, W. Weckesser, J. Bright, *et al.*, "Scipy 1.0: fundamental algorithms for scientific computing in python," *Nature methods*, vol. 17, no. 3, pp. 261–272, 2020.

[22] T. A. Davis, *Direct methods for sparse linear systems*. SIAM, 2006.

[23] H. Group *et al.*, "Hierarchical data format, version 5," *Zenodo*, 1997.

[24] H. He and E. A. Garcia, "Learning from imbalanced data," *IEEE Transactions on knowledge and data engineering*, vol. 21, no. 9, pp. 1263–1284, 2009.

[25] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "Smote: synthetic minority over-sampling technique," *Journal of artificial intelligence research*, vol. 16, pp. 321–357, 2002.

[26] R. Caruana, "Multitask learning," *Machine learning*, vol. 28, no. 1, pp. 41–75, 1997.

[27] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, *et al.*, "Pytorch: An imperative style, high-performance deep learning library," *Advances in neural information processing systems*, vol. 32, 2019.

[28] M. Fey and J. E. Lenssen, "Fast graph representation learning with pytorch geometric," *arXiv preprint arXiv:1903.02428*, 2019.

[29] M. Ott, S. Edunov, D. Grangier, and M. Auli, "Scaling neural machine translation," in *Proceedings of the Third Conference on Machine Translation: Research Papers*, pp. 1–9, 2018.

[30] P. Micikevicius, S. Narang, J. Alben, G. Diamos, E. Elsen, D. Garcia, B. Ginsburg, M. Houston, O. Kuchaiev, G. Venkatesh, *et al.*, "Mixed precision training," *arXiv preprint arXiv:1710.03740*, 2017.

[31] P. J. Huber, "Robust estimation of a location parameter," in *Breakthroughs in statistics: Methodology and distribution*, pp. 492–518, Springer, 1992.

[32] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *arXiv preprint arXiv:1412.6980*, 2014.

[33] R. Pascanu, T. Mikolov, and Y. Bengio, "On the difficulty of training recurrent neural networks," in *International conference on machine learning*, pp. 1310–1318, Pmlr, 2013.